

بخش اول:

فایل `turing_machine.py`:

در تابع `run` اول چک میشه اگه `tape` ورودی خالی بود یه دونه از `symbol_blank` میذاره توش (خود `tape` هم که یه لیست پایتون هست) بعد هم سمت چپ `tape` رو یه لیست خالی میذاره و سمت راست رو `tape[1:]` میذاره بعد میره داخل حلقه اصلی اگه حالت فعلی پذیرش یا رد بود `action` رو با مقدار مناسب مقداردهی میکنه بعد هم متغیر `config` رو با مقدارهای مربوطه پر میکنه و این دوتا رو `yield` میکنه بعد میره چک میکنه اگه به حالت پذیرش یا رد رسیده بود که میاد بیرون از حلقه و قبلش هم رشته نهایی رو پرینت میکنه.

در صورت ادامه میره و از دیکشنری انتقال مربوط به حالت فعلی و چیزی که داخل خانه ی فعلی `tape` هست رو برمیذاره و اگه انتقالی براش نبود میره به حالت `reject`. اگه هم انتقال تعریف شده بود مقادیر `state` و `symbol` رو با مقادیر جدید به روزرسانی میکنه و بعد میره برای چک کردن جهت حرکت. اگه حرکت `R` بود `symbol` به روز شده رو به لیست سمت چپ اضافه میکنه و اگه لیست سمت راست خالی بود حالت `symbol` رو دوباره به روز میکنه و مقدار `blank_symbol` رو میذاره براش ولی اگه لیست سمت راست خالی نبود اولین خونه‌ش رو `pop` میکنه و میذاره تو `symbol`. برای حرکت `L` هم همین اتفاقا برعکسش میفته با این تفاوت که اگه لیست سمت چپ خالی بود، درصورتی که تابع صرفاً برای چک کردن پذیرش (منظور تابع `accept` هست) و یا توسط کد های بخش 3 فراخوانی نشده بود یه دونه لاگ هشدار میندازه و ادامه میده (برای توقف هم میشه همینجا `state` رو با مقدار `reject` به روزرسانی کرد که داخل کد این بخش کامنت شده که اگه کامنت برداشته بشه هرموقع که `head` خواست از سمت چپ بیشتر بره متوقف میشه محاسبه

در تابع `accept` تابع `run` اجرا میشه و روی هر `yield` از `run` چک میشه اگه به حالت های پذیرش یا رد رسیدیم `true` یا `false` خروجی میده و یا اگه `limit` تعداد انتقال ها رد شد هم خروجی `false` رو میده

تابع `reject` هم همون تابع `accept` رو اجرا میکنه و خروجیش رو `not` میکنه و `return` میکنه

تابع `debug` هم میاد دوباره `run` رو اجرا میکنه ولی این دفعه میاد روی هر `yield` مقادیر رو `print` میکنه تا داخل ترمینال مرحله به مرحله مشخص باشه. متغیر `last_time` هم برای این هست که تابع برای بار آخر صدا زده بشه و `yield` انجام بشه تا اون قسمتی که داخل تابع `run` مربوط به پرینت رشته نهایی هست هم انجام بشه

(الف)

Example1: `q0, saw_#, saw_##, qa`

Example2: `q0, FindDelimiter0, Check0, FindDelimiter1, Check1, FindLeftmost, FindNext, qa, End`

(ب) داخل خود تابع `debug` فایل ماشین تورینگ با هر `yield` مقادیر `configuration` هم پرینت میشه

(ج) فایل `example1` فقط رشته `##` رو قبول میکنه و فایل `example2` رشته هایی به فرمت `w#w` رو که `w` خودش رشته ای 0 و 1 هست رو قبل میکنه

بخش دوم:

عملیات جمع: خب اول ماشین شروع میکنه تا برسه به اون 0 وسط. اون 0 رو یک میکنه و میره به آخر رشته. وقتی رسید به خونه خالی (آخر رشته) یکی برمیگرده عقب تا بیاد روی آخرین 1 و اون یک رو تبدیل به 0 میکنه و میره به حالت پذیرش

عملیات ضرب: به ازای هر 1 قبل 0 جداکننده ماشین اون 1 رو پاک میکنه میره به شروع 1 های بعد 0 میانی. به ازای هر یک ۷ میذاره میره آخر بعد میره آخر رشته یک Z اضافه میکنه دوباره برمیگرده تا به آخرین ۷ که رسید یکی میره جلو اگه 1 بود که دوباره ۷ میذاره میره آخر یه Z دیگه اضافه میکنه و دوباره برمیگرده ولی اگه Z بود یعنی یه دور به ازای همه یک های بعد 0 Z گذاشته پس شروع میکنه به reset کردن ۷ ها به همون مقدار اولیه 1 و بعد دوباره میره به اول رشته. اگه اول رشته 1 بود دوباره همین روند تکرار میشه ولی اگه 0 بود یعنی به ازای همه یک ها یک بار این روند اجرا شده پس شروع میکنه و اون 0 و همه 1 های بعدش رو پاک میکنه فقط Z ها بمونن. از اونجا که به ازای هر 1 قبل 0 میره و به ازای هر یک بعد 0 یه دونه Z اضافه میکنه پس تعداد این Z ها همیشه نتیجه ضرب. در نتیجه در آخر ماشین همه Z ها رو با 1 جایگزین میکنه و میره به حالت پذیرش

بخش سوم:

الف) خب اول وقتی میخوایم رکورد بیدستر پرتلاش رو پیدا کنیم عملاً با یه کار غیرممکن طرفیم. مشکل اول اینه که فقط با اضافه شدن یه دونه کارت جدید تعداد ماشین های ممکن خیلی وحشتناک زیاد میشه مثلاً برای ماشینی که n تا کارت داره تعداد کل حالت ها از فرمول $(4(n+1))^{2n}$ به دست میاد که تست کردن همه اینا حتی از توان قوی ترین ابرکامپیوترها هم خارجه. ولی دردسر اصلی که باعث میشه رسماً غیرقابل حل باشه اینه که گره خورده به مسئله توقف تورینگ. یعنی ما برای پیدا کردن رکورددار چاره ای نداریم جز اینکه برنامه هارو اجرا کنیم و منتظر بمونیم تا متوقف بشن ولی خب از کجا بفهمیم ماشینی که مثلاً یه میلیون بار کار کرده بالاخره یه جا وایمیسته یا تا ابد تو یه حلقه گیر افتاده؟ هیچ راهی برای فهمیدن این نیست. در واقع اگه میتونستیم رکورد بیدستر رو دقیق حساب کنیم مسئله توقف هم خودش حل میشد چون راحت میگفتیم اگه ماشینی بیشتر از این رکورد کار کرد پس حتماً تو حلقه گیر کرده و دیگه متوقف نمیشه. پس از اونجا که تورینگ ثابت کرده پیش بینی توقف برنامه ها کلاً راه حلی نداره در نتیجه پیدا کردن قطعی بیدستر پرتلاش هم به همون اندازه ناممکن و نشدنی.

- ****Undecidability****: There is no general algorithm to determine whether an arbitrary Turing machine will halt for a given input
- ****Non-computability****: The Busy Beaver function $BB(n)$ is non-computable - no Turing machine can compute it for all n
- ****Rapid growth****: $BB(n)$ grows faster than any computable function
- $BB(1) = 1$
- $BB(2) = 6$
- $BB(3) = 21$
- $BB(4) = 107$
- $BB(5) = 47,176,870$
- $BB(6) > 10^{36},534$

اینم داخل خود فایل md بود میشد به عنوان جواب آورد

ب) همون فایل رو همیشه استفاده کرد چون گسترش از هر دو طرف رو هندل میکنه و برای درخواست غیر بخش 3 یک لاگ هشدار پرینت میکنه

```
# Python generators are useful for Turing machines because they invert control  
# over to the caller. They can generate an output stream lazily  
# Iterators allow lazy evaluation, only generating the next element of an iterable object when requested.  
# This is useful for very large data sets.
```

```
# EXAMPLE:
```

```
# >>> x = (i ** 2 for i in range(1, 10))
```

```
# >>> next(x)
```

```
# 1
```

```
# >>> next(x)
```

```
# 4
```

```
# >>> next(x)
```

```
# 9
```

```
# This means that you could construct a Turing Machine lazily, repeatedly
```

```
# ask for output with next, _only when needed_, or collect all its outputs
```

```
# if needed
```

```
# EXAMPLE:
```

```
# >>> x = (i ** 2 for i in range(1, 10))
```

```
# >>> list(x)
```

```
# [1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
# generator could write a function that
```

```
#     potentially runs forever and it's up to the caller to decide how many
```

```
#     steps are executed. We give the user control to execute us one step at a time. This
```

```
#     is what Python generators are (partially) for. The yield expression
```

```
#     suspends us and gives control to the caller until he or she decides to
```

```
#     resume our execution.
```

```
# Because execution is done in a generator, it's possible to have infinite executions
```

```
#     but the acceptance checks are limited by the number of steps they are allowed to perform.
```

د) واقعا نیازی به بانویسی نیست. با همون فایل ماشین تورینگ فقط انتقال های ماشین پرتلاش رو بهش میدیم تا اجرا کنه روی رشته. تعداد نماد یک هم 4 شد. همون ماشین تورینگ نمونه انتقال هاش رو دادم به تابع run و اوکی بود

ه) خب تعداد 1 ها که بیشترین ممکن بود مثل بقیه ماشین های پرتلاش 2 کارتی خب پس خروجی هم باید بهترین ممکن ارزیابی بشه طبیعتا (و) خب از اونجایی که گفته شد لازم نیست حتما بهترین باشه برای 3 کارتی صرفا یه کارت اضافه کردم به جای یکی از 0 ها 1 بذاره و بره به حالت پذیرش و حالت پذیرش 2 کارتی رو گذاشتم این حالت سومی که اضافه کردم برای 4 کارتی هم به همین 3 کارتی با همین روش یک کارت اضافه کردم پس 3 کارتی 5 تا 1 در نتیجه داره و 4 کارتی 6 تا 1 در رشته نهایی داره

م) صادقانه نه – همون بهترین ماشین 4 کارتی رو یه دونه کارت با همون روش قسمت و اضافه کردم. برای جدید بودنش هم بعید میدونم جدید باشه چون این ایده طبیعتا به ذهن خیلیا اومده

ک) قطعا برنامه های من از بهترین های شناخته شده بهتر نیستند – درواقع اونطوری که داخل سایت نوشته شده بود برنامه های فعلی بهترین های ممکن هستند و دیگه از این بهتر نمیشه. انتقال های بهترین برنامه برای هر 2 تا 5 کارته داخل کد هست